RepoPilot

An AI teammate for open-source software projects

A detailed, beginner-friendly project guide — written for readers who may know HTML and CSS but have never used GitHub.

The idea in one sentence

RepoPilot connects to a public GitHub project, understands a reported software problem, asks Gemma 4 to investigate the code, prepares a possible fix, tests that fix in a separate Docker environment, and — after a human approves it — can prepare a GitHub pull request.

Important: RepoPilot is an assistant for developers. It does not get permission to change somebody else's project automatically, and it must never claim that a fix works unless tests were actually run.

Project direction

AI model	Gemma 4 API
Application	React + Node.js + Express
Database	MongoDB
Code testing	Docker sandbox
Source-code platform	Public GitHub repositories
Cloud deployment	DigitalOcean
Development target	Solo developer, rapid hackathon MVP

1. Start with the problem: why does RepoPilot exist?

Imagine that someone builds a free website, tool, or library and shares its source code so other people can use it. After a while, a user reports: “The upload button crashes when I select a large file.” The project maintainer now has to understand the report, find the right part of the code, reproduce the error, write a fix, test it, and review the change.

That work can take hours. A beginner who wants to help may not even know which file to open. RepoPilot is designed to make that journey easier.

A simple analogy

Think of a software project as a large school building. Its rooms are folders, the objects inside the rooms are files, and the instructions for how the building works are its code. A bug report is like a note saying, “The lights in Room 12 do not turn on.” RepoPilot is a trained assistant who reads the building map, visits the likely room, investigates the problem, suggests a repair, and checks whether the lights work afterwards.

The project has two main beneficiaries:

- Maintainers: people responsible for keeping a software project working. RepoPilot can help them investigate and prepare fixes.
- Contributors: people who want to improve a project. RepoPilot can explain unfamiliar code and guide them towards a useful contribution.

RepoPilot does not replace the maintainer. It prepares work that a person can inspect and approve.

2. What is GitHub? (No previous experience needed)

GitHub is a website where developers store, share and work together on source code. It is a little like Google Drive for software projects, but it also keeps a detailed history of changes and provides tools for reporting problems and reviewing proposed updates.

GitHub term	Plain-English meaning	Everyday comparison
Repository (repo)	The complete project: its files, folders and history.	A shared project folder.
Issue	A report of a bug, question or requested improvement.	A support ticket or task card.
Commit	A saved set of code changes with a message explaining them.	A named save point.
Branch	A separate line of work where changes can be made safely.	A copy of a document for editing.
Pull request (PR)	A proposal asking the project owners to review and accept changes.	Suggesting edits to a shared document.
Fork	Your own copy of somebody else's GitHub repository.	Making your own copy of a shared folder.

GitHub term	Plain-English meaning	Everyday comparison
Merge	Combining approved changes into the main project.	Accepting suggested edits into the original document.
README	A document explaining what the project does and how to use it.	The project’s instruction booklet.

Example: A project has an issue titled “Login fails when the password contains a special character.” A contributor can investigate the code, make a change on their own branch, and open a pull request. The project maintainer reviews the proposal. If it is good, they can merge it.

You do not need to know GitHub before understanding RepoPilot. The important idea is that GitHub contains projects, reports about problems, and a safe process for suggesting improvements.

3. What exactly will RepoPilot do?

The main feature is called Issue-to-PR. It means helping turn a reported problem into a proposed code contribution. Here is the journey in simple steps.

Step 1 — Connect a public project

The user pastes a public GitHub repository address, such as github.com/owner/project. RepoPilot retrieves basic information: project name, description, main programming language, README and file/folder list.

Step 2 — Choose a reported problem

RepoPilot retrieves the project's open issues. The user selects one issue. An issue is simply a report or task. For example: “The application crashes when a user uploads a large image.”

Step 3 — Understand the report

Gemma 4 reads the issue title and description. It identifies what is going wrong, what the expected behaviour might be, and what information is missing. It can classify the issue as a bug, feature request, documentation task or question.

Step 4 — Investigate the source code

RepoPilot searches the repository for files that may be related to the issue. It reads only the relevant files rather than blindly sending the entire project to the AI. It can inspect filenames, functions, imports, tests and documentation.

Step 5 — Explain the likely cause

Gemma 4 receives the issue and relevant code. It prepares a reasoned explanation of where the problem may be and why. This is a hypothesis, not proof. The developer can inspect the files and explanation.

Step 6 — Propose a code change

The AI suggests a patch. A patch is a small, structured description of exactly which lines should be added, removed or changed. RepoPilot checks that the patch refers to permitted files and does not escape the temporary project folder.

Step 7 — Test the proposed change

RepoPilot applies the patch to a temporary copy of the repository and runs approved test commands inside Docker. It records the command, output and exit status. A passing test is useful evidence, but it does not prove that every possible bug is fixed.

Step 8 — Prepare a pull request

If the user approves, RepoPilot can create a branch in the user's own fork, commit the changes and open a pull request against the original project. The original project owner decides whether to accept it.

Step 9 — Produce an evidence report

RepoPilot displays the issue, files changed, explanation, code difference, test commands, test results and pull-request link. This makes the work easier to review.

The complete journey

1. Public GitHub project

↓

2. Select an open issue

↓

3. Gemma 4 investigates relevant code

↓

4. Generate a patch

↓

5. Run tests in Docker

↓

6. Human reviews and approves

↓

7. Create a pull request

4. What is Gemma 4, and why does RepoPilot need it?

Gemma 4 is an AI model family from Google. An AI model is software trained to understand and generate information. Depending on the model and the way it is accessed, it can work with text, images, reasoning tasks and code-related requests.

In RepoPilot, Gemma 4 acts like the investigator and problem-solving assistant. It is not the database, not GitHub, and not the computer that runs tests. It analyses information and recommends what should happen next.

A simple comparison

Imagine a mechanic. The customer describes a car problem, the mechanic reads the service manual, inspects the car, recommends a repair and asks a technician to perform checks. Gemma 4 is the reasoning mechanic. GitHub provides the project information. RepoPilot's backend coordinates the work. Docker is the safe test workshop.

RepoPilot task	Gemma 4's role
Understand an issue	Summarise the problem and identify missing details.
Find likely code locations	Suggest search terms, functions and files to inspect.
Explain the cause	Reason from the issue and retrieved source code.
Suggest a fix	Generate a structured code patch.
Read a screenshot	Interpret an attached error screenshot if the chosen model endpoint supports image input.
Use tools	Request a specific backend action using a defined tool format, when supported.

Does the AI directly control the computer?

No. This is a crucial safety decision. Gemma 4 can request an action, but the Node.js backend decides whether that action is allowed. For example, the model may request that a file be read. The backend checks the path, reads the file and returns its contents. The model does not receive unrestricted access to the server.

Why use an API instead of running the model on our own computer?

An API lets our application send a request to a hosted model service and receive a response. This is quicker to set up for a one-day hackathon than downloading model weights, configuring inference software and managing hardware. The exact Gemma 4 model name, API endpoint, supported input formats and tool-calling format must be checked against the provider's current documentation before integration.

A model response can be wrong. RepoPilot must validate its output, inspect patches, run tests and show uncertainty honestly.

5. What technologies are used, explained for beginners

You do not need to learn every technology deeply before understanding the product. Each part has one job. Think of RepoPilot as a small company: the user interface is the reception desk, the backend is the operations team, the AI is the analyst, the database is the filing cabinet, and Docker is the secure testing room.

Technology	What it is	What it does in RepoPilot
HTML	The structure of a web page.	Provides the basic content and elements in the browser.
CSS / Tailwind	The visual styling of a page.	Controls layout, spacing, colours and responsive design.
JavaScript	A programming language used on websites and servers.	Makes buttons, forms and application behaviour work.
React	A JavaScript library for building user interfaces.	Builds the dashboard, issue list, progress timeline and report screens.
Node.js	A runtime that runs JavaScript outside the browser.	Runs the backend application.
Express	A framework for Node.js web servers.	Creates API routes that the frontend calls.
TypeScript	JavaScript with a type system.	Helps catch certain mistakes while developing.
MongoDB	A database that stores documents.	Stores users, repositories, agent runs and reports.
Gemma 4 API	A way to send tasks to the AI model.	Analyses issues and code, and proposes changes.
GitHub API / Octokit	A way for software to communicate with GitHub.	Reads repository data and performs authorised GitHub actions.
Docker	A tool for running software in isolated containers.	Runs repository tests in a separate, controlled environment.
DigitalOcean	A cloud infrastructure provider.	Hosts the application and supporting services online.

How the parts talk to each other

The browser sends a request to the Express backend. The backend retrieves data from GitHub or MongoDB, and sends relevant information to Gemma 4 when AI analysis is needed. If a patch needs testing, the backend asks the Docker worker to run an approved test. The results are saved and sent back to the browser.

What is an API?

An API is a defined way for two software systems to communicate. Think of it as a waiter in a restaurant: you make a request from a menu, the waiter passes it to the kitchen, and brings the result back. The frontend does not need to know how the backend performs every task.

What is a database?

A database stores information so the application can retrieve it later. RepoPilot may store which repository was analysed, which issue was selected, what the agent found, what patch was proposed and what test results were produced. It should not store private access tokens in plain text.

6. Docker: the safe testing room

A repository may contain code written by people we do not know. Running that code directly on the main application server would be dangerous. A test command could delete files, consume too much memory, or attempt to access sensitive information.

Docker lets us run a program inside a container: an isolated environment with its own filesystem and controlled resources. It is like giving a visitor a separate workshop instead of letting them freely enter the entire building.

The intended test process

- Create a temporary copy of the selected repository.
- Apply the proposed patch only inside that temporary copy.
- Start a preconfigured, non-privileged test container.
- Run an allow-listed test command, such as a known npm test script.
- Capture the output, exit code and duration.
- Stop and remove the container and temporary workspace.

Safety rules

- Do not run containers in privileged mode.
- Do not mount the host's Docker socket into the test container.
- Do not pass GitHub tokens, API keys or production secrets into the container.
- Disable network access for tests unless a specific, reviewed need exists.
- Set CPU, memory, process and time limits.
- Run as a non-root user where possible.
- Do not automatically run a Dockerfile supplied by an unknown repository.
- Check every generated file path and reject attempts to write outside the workspace.

Honest reporting: “Tests passed” means the selected commands completed successfully in the configured environment. It does not mean the software is guaranteed to be bug-free. If tests were not run, the report must say “Not tested.”

7. How GitHub permissions and pull requests work

A public repository can be read by anyone, but that does not mean everyone can change the original project. RepoPilot should respect the owner's permissions.

For the initial workflow, the user connects their GitHub account. RepoPilot creates or uses a fork — the user's own copy — then creates a separate branch, adds the proposed change and opens a pull request to the original project.

A pull request is a request for review, not an automatic acceptance. The original maintainers can comment, ask for changes, reject it or merge it. RepoPilot must never promise that a pull request will be accepted.

The user must explicitly approve the proposed changes before RepoPilot publishes a branch or opens a pull request. Access tokens stay on the server, are protected, and are never included in AI prompts.

8. What makes RepoPilot different?

The project should not be presented as “a chatbot that can read GitHub.” Its value is the complete, reviewable workflow. Three features help make that clear.

A. Repository understanding

RepoPilot reads the project's README, folder structure and selected source files. It uses the issue to decide which files are relevant. This is more useful than sending random files to an AI model and hoping it understands the project.

B. Issue-to-PR workflow

The agent moves through investigation, proposed fix, patch validation, testing and pull-request preparation. The interface should show each stage and the real result. This is the central feature to demonstrate to judges.

C. Patch Evidence Report

Every run produces a report that answers: What issue was selected? What files were examined? What changed? Why was the change suggested? Which test command ran? Did it pass? Is there a pull-request link? This gives reviewers evidence instead of asking them to trust a vague AI claim.

Optional extra: First Contribution Copilot

A beginner can choose a language or skill level and ask RepoPilot to find a manageable issue. The tool can explain the issue in simple words, identify relevant files and provide a learning path. This is a useful addition if the core workflow is already reliable.

What RepoPilot will not try to do in the first version

- It will not support every programming language on day one. The first reliable runner should focus on Node.js projects.
- It will not automatically merge pull requests.
- It will not run untrusted code on the main application server.
- It will not guarantee that every AI-generated patch is correct.
- It will not require blockchain. Blockchain does not solve the core problem in this version.

9. What the user sees: a simple product walkthrough

Screen	What appears on the screen	Why it matters
Dashboard	A box to paste a public GitHub repository URL, plus recently analysed projects.	The user knows where to start.
Repository page	Project name, description, language, README summary and open issues.	The user can understand the project.
Issue page	Issue title, description, labels and an “Investigate” action.	The user selects a concrete problem.
Agent activity	Steps such as reading files, searching code, analysing and preparing a patch.	The process is visible rather than hidden.
Patch review	Before-and-after code changes, files affected and AI explanation.	The user can inspect what will change.
Test results	Commands executed, output, pass/fail status and time taken.	The user sees actual validation evidence.
Evidence report	Summary of the investigation, patch, tests and PR link.	The work is easy to review and share.

The most important screen is the agent activity and patch review. Judges should see real actions and real outputs, not a decorative progress animation.

10. The system architecture in plain language

The architecture is the plan showing how the different pieces of RepoPilot connect. The diagram below is a simplified view of the proposed system.

USER'S BROWSER	React dashboard: repository input, issue list, agent progress, patch review and report.
EXPRESS BACKEND	Receives requests, checks permissions, coordinates GitHub, Gemma, database and Docker.
GEMMA 4 API	Analyses issue and relevant code; proposes a cause, patch or next action.
GITHUB	Provides public repository information, issues, forks, branches, commits and pull requests.
DOCKER WORKER	Runs selected tests in a temporary isolated container and returns output.
MONGODB	Stores users, repository metadata, run status and evidence reports.
DIGITALOCEAN	Hosts the web application and backend; can store generated artefacts in cloud storage.

Why use one agent orchestrator?

The first version will use one backend-controlled workflow with several stages: issue analysis, code investigation, patch generation, test validation and pull-request preparation. These stages can use different prompts and structured outputs, but they do not need to be separate autonomous agents. This is simpler to build, test and explain in a one-day hackathon.

How progress reaches the screen

The backend can send progress updates to the browser using Server-Sent Events (SSE). SSE is a one-way live connection: the server can tell the browser “Reading files”, “Generating patch” or “Tests completed” without the browser repeatedly asking for updates.

Why DigitalOcean?

DigitalOcean is the cloud provider selected for hosting the application. The hackathon version can use it to run the frontend and backend and, if time allows, store reports or other files in cloud object storage. Gemma 4 inference is planned through its supported API, so a GPU server is not required for the initial version.

The exact deployment arrangement must be tested. A Docker execution worker needs access to a Docker runtime; do not assume every application-hosting service permits launching nested containers. If the chosen hosting plan cannot safely run the worker, deploy the worker separately on a suitable host.

11. The one-day hackathon plan

The deadline is very short, so the project should be built in a strict order. The aim is to get one complete workflow working before adding optional features.

Target time	Build task	Proof that the stage works
Hours 0–2	Create frontend and backend, configure TypeScript, environment variables, MongoDB and Docker.	The app starts, backend responds, database connects and Docker runs a test container.
Hours 2–4	Connect GitHub and retrieve public repository details and issues.	Paste a repository URL and see its real open issues.
Hours 4–7	Connect Gemma 4 and implement structured issue analysis.	Select an issue and receive a structured analysis.
Hours 7–11	Retrieve relevant files and generate a validated patch.	See a real code diff generated for a prepared issue.
Hours 11–14	Run a predefined test command in Docker.	See actual test output and exit status.
Hours 14–17	Create a branch in a fork and prepare a pull request after approval.	A real PR is opened or a clearly labelled fallback patch is available.
Hours 17–20	Polish dashboard, agent timeline and evidence report.	A judge can understand the workflow without developer explanation.
Hours 20–22	Deploy and configure the cloud environment.	The deployed app completes the main workflow.
Hours 22–24	End-to-end testing, README, demo recording and pitch.	A repeatable demo and complete submission.

The must-have version

If time runs short, preserve these five working capabilities: (1) retrieve real public GitHub issues, (2) use Gemma 4 to analyse one issue and relevant files, (3) generate a patch, (4) run a real test in Docker, and (5) show a clear evidence report. A real PR is the target, but do not fake one if permissions or time prevent it.

Features to postpone

- Private repositories and organisation-level access.
- Support for Python, Java, Go, Rust and every other runtime.
- Complex vector search and full repository indexing.
- Multiple independent agents and team collaboration.
- Blockchain credentials, automatic merging and advanced analytics.

12. The judging demo: show one real journey

A short, reliable demonstration is better than a long list of unfinished features. Prepare a public repository with a known, reproducible issue and a test that can demonstrate the intended fix. Confirm that you have permission to create a fork and pull request from the account used for the demo.

Demo time	What to show	What to say
0:00–0:25	Paste a public repository URL.	“RepoPilot starts by understanding a real open-source project.”
0:25–0:50	Show real open issues and select one.	“We choose a reported problem instead of asking a generic question.”
0:50–1:40	Show Gemma 4 investigating relevant files.	“The model reasons over retrieved project context, while our backend controls access to tools.”
1:40–2:15	Show the generated patch and explanation.	“The output is a proposed code change that a developer can inspect.”
2:15–2:50	Show Docker test output.	“We validate the patch in an isolated environment and report the real result.”
2:50–3:25	Show evidence report and approved PR.	“The final contribution is reviewable by the maintainers. RepoPilot does not merge it automatically.”
3:25–3:50	Show the deployed application and architecture.	“Gemma 4 provides intelligence; DigitalOcean hosts our application; Docker provides isolated testing.”

Prepare for a failed network call

Keep a backup screen recording of the same workflow. The live demo should still use real analysis and testing where possible. If a service fails, explain what happened and show the recorded result rather than presenting a fake live success.

13. How to explain RepoPilot to judges who are not technical

Use this simple explanation:

> *“Imagine that a free software project receives a message saying that one of its features is broken. Usually, a developer has to read the report, search through the code, find the problem, write a repair and test it. RepoPilot helps with those steps. It uses Gemma 4 as an AI investigator, retrieves the relevant code from GitHub, suggests a repair, tests that repair in a separate safe environment and prepares a change for the project owner to review. It does not secretly change somebody else's software. A human remains in control.”*

One-line explanation

RepoPilot is like an AI assistant that helps software teams find problems, prepare repairs and prove what was tested before asking a human to approve the change.

14. Frequently asked questions

Do I need to know GitHub to use RepoPilot?

No. The interface should guide the user through connecting a public repository, choosing an issue and reviewing the proposed fix. Some GitHub concepts are useful, but the guide and interface explain them.

Does RepoPilot write the entire application?

No. It investigates a specific issue and proposes a focused change. A developer still reviews the change and decides whether to publish it.

Can it fix every bug?

No. Some issues need more context, special hardware, external services or human judgement. RepoPilot should say when it cannot confidently investigate an issue.

Does a passing test prove the patch is correct?

No. It proves only that the selected tests passed in the configured environment. Other tests or real-world conditions may still reveal a problem.

Can RepoPilot change the original public repository directly?

Not unless the user has permission. The planned workflow creates a branch in the user's fork and opens a pull request to the original repository.

Why is Docker needed?

It provides a separate environment for running project tests so untrusted code is not executed directly on the main application server.

Why use Gemma 4?

It is the selected AI model for understanding issue descriptions, analysing code context, reasoning about possible causes and generating proposed changes. Its exact API features must be verified before implementation.

Why use DigitalOcean?

It is the selected cloud platform for deploying the application and supporting infrastructure. The project should demonstrate real cloud hosting, not merely display a DigitalOcean logo.

Why no blockchain?

Blockchain is not necessary to solve the core problem. Adding it would increase complexity without improving the central issue-to-PR workflow.

What happens if the AI is wrong?

The backend validates its outputs, the patch is displayed for review, tests are run in a sandbox and a human must approve publishing. The evidence report must show failures honestly.

15. Final project summary

RepoPilot is an AI-powered open-source engineering assistant. It helps developers and beginners understand public GitHub repositories, investigate reported issues, prepare code changes, test those changes in an isolated Docker environment and create review-ready pull requests with human approval.

Area	Final decision
Project name	RepoPilot
Tagline	From GitHub issue to verified, review-ready pull request.
Primary AI	Gemma 4 API
Frontend	React, TypeScript and Tailwind CSS
Backend	Node.js, Express and TypeScript
Database	MongoDB
GitHub integration	Public repositories through GitHub API / Octokit
Execution	Docker sandbox, initially focused on Node.js projects
Cloud	DigitalOcean
Blockchain	Not included in the hackathon MVP
Core differentiator	Issue-to-PR workflow with Patch Evidence Report
Human control	Explicit approval before publishing a pull request

The principle to remember

Do not measure RepoPilot by how many AI features it claims to have. Measure it by whether it can take one real issue, investigate the real code, propose a useful patch, run real tests and present honest evidence for a human reviewer.

Prepared as a beginner-friendly project explanation and implementation reference. Technical integrations, model availability, API permissions and cloud deployment details should be verified against the providers' current documentation before production use.